

EviRCA: Decoupling Evidence Extraction from Reasoning for Microservice Root-Cause Analysis

Yuhao Wang¹, Zhen Qin^{2*}, Xingliang Wang¹, Guochang Li¹, Weize Li³, and Shuiguang Deng^{1*}

¹College of Computer Science and Technology, Zhejiang University, Hangzhou, China

²School of Software Technology, Zhejiang University, Ningbo, China

³China Telecom Cloud Computing Corporation, Beijing, China

22421122@zju.edu.cn, zhenqin@zju.edu.cn, wangxingliang@zju.edu.cn, gcli@zju.edu.cn,

liweize@chinatelecom.cn, dengsg@zju.edu.cn

*Corresponding authors: Zhen Qin and Shuiguang Deng.

Abstract—Root-cause analysis (RCA) is a critical yet labor-intensive task for maintaining modern microservice systems, making it an attractive target for applying large language models (LLMs). Recent agentic approaches allow an LLM to iteratively explore raw telemetry by generating and executing code. However, this design asks a single model to simultaneously retrieve evidence, localize faults, and infer root causes over large volumes of heterogeneous telemetry, leading to high computational cost, unstable behavior, and limited diagnostic accuracy. However, the telemetry data consists of numeric metrics, structured traces, and machine-generated logs, which are not appropriate for being directly processed by LLMs. Therefore, this work presents EviRCA, a framework for LLM-based RCA that decouples deterministic evidence extraction from LLM reasoning. A system-agnostic extraction stage converts raw metrics, traces, and logs into a compact set of faithful multimodal evidence cards, while the LLM reasons only over these structured observations through a small set of predefined read-only tools, without accessing raw telemetry or executing code. We evaluate EviRCA on OpenRCA, a benchmark built from real, heterogeneous telemetry across three enterprise systems. EviRCA achieves a Correct rate of 40.6–43.9% across two different LLMs, substantially outperforming prior OpenRCA baselines that achieve up to 15.2%, while reducing token consumption by 15–26× and execution time by 3–20×. Moreover, EviRCA successfully solves hard cases requiring simultaneous processing of time, components and reasons, a setting in which previous approaches reported near-zero performance. Our process-level failure analysis further shows that the binding difficulty lies in judging evidence that the extraction stage has already surfaced, instead of searching for it. This suggests that the effectiveness of LLM-based RCA highly depends on the quality of evidence extraction.

Index Terms—root cause analysis, AIOps, large language models, microservices, observability

I. INTRODUCTION

Cloud-native microservice architectures now underpin much of the software that businesses and users depend on [1]. However, their scale and constant change make failures frequent and costly, demanding continuous on-call attention while risking substantial financial and reputational damage. When a service degrades, on-call engineers must perform root-cause analysis (RCA) to trace user-visible symptoms back to the faulty component. This process requires inspecting massive, heterogeneous telemetry data, including metrics, logs, and traces. While traditional RCA algorithms, such as call-

graph random walks, causal-graph construction and log-trace analysis, can be accurate, they heavily rely on pre-specified topologies, curated training data, or system-specific modeling, limiting their transferability to new deployments [2], [3].

The emergence of large language models (LLMs) has fundamentally reshaped many software engineering tasks. In domains such as code generation, issue resolution, and testing, agentic systems and benchmarks such as SWE-bench [4], SWE-agent [5], OpenHands [6], and MetaGPT [7] have driven rapid progress, and tools such as Copilot [8], Codex [9], and Claude Code [10] have integrated into daily development. These successes share a common basis: their inputs are natural-language-friendly, such as source code and issue descriptions, and their outputs admit cheap, executable verification through tests. This capacity for natural-language interaction also makes LLMs highly attractive for RCA, allowing engineers to state diagnostic goals intuitively without worrying about environment-specific algorithms. Early LLM-based RCA approaches primarily treated the model as a text summarizer or classifier over incident reports, focusing on predicting broad root-cause categories or suggesting mitigations [11], [12], [13]. These outputs, however, remain at the level of textual summarization or coarse category prediction, without localizing the fault in raw, live telemetry. OpenRCA [14] provides a concrete formulation of the problem by introducing a goal-driven, reproducible benchmark built on heterogeneous telemetry collected from three enterprise systems, in which a method must recover a queried subset of a fault’s time, component, and reason. Its reference method, RCA-Agent, gives the LLM agency over the telemetry, letting it repeatedly write and run Python over the raw data in the ReAct style [15] until it commits an answer. Yet, despite this operational freedom, the strongest LLM evaluated in the original study solves a mere 11.3% of cases [14].

This limited performance reflects a fundamental mismatch between LLMs and the nature of telemetry data. Raw telemetry consists of high-volume, low-level signals including numerical metrics, structured traces, and machine-generated logs, which provide little semantic abstraction for language models. Unlike natural language or source code, telemetry cannot be directly processed through pretrained linguistic representations, nor

can conclusions be validated using inexpensive executable tests. Consequently, requiring a single agent to detect anomalies, localize, and attribute over this raw telemetry within one loop is prohibitively expensive and brittle. Such an unconstrained loop can easily consume nearly a million tokens per incident, which may produce hallucinated data points and schema fields while causing latency to grow with every step. Furthermore, the traditional end-to-end evaluation protocol, which counts a prediction as correct only when every queried field is recovered exactly, obscures the underlying source of failure. A single pass/fail metric cannot distinguish failures caused by inadequate evidence retrieval from those caused by incorrect reasoning over evidence that has already been obtained. Concurrent studies corroborate this conclusion: agentic RCA failures are largely shared across model capability tiers and rooted in the agent architecture itself, and end-to-end scoring conflates root-cause reasoning with unrelated skills such as anomaly detection and code generation [16], [17].

Goal-driven RCA naturally decomposes into three subtasks: 1) anomaly detection over heterogeneous telemetry, 2) candidate localization, and 3) semantic discrimination of the true root cause and its underlying failure reason. In fact, the first two are deterministic signal-processing problems that can be solved reliably and reproducibly without an LLM, whereas only the final stage requires semantic reasoning. We therefore argue that *goal-driven RCA should focus not on discovering evidence through unconstrained exploration, but on accurately interpreting evidence once it has been retrieved*. This motivates a more effective division of labor: deterministic components first compress raw telemetry into compact, faithful evidence, allowing the LLM to focus exclusively on high-level diagnostic reasoning.

Guided by this insight, we present EviRCA, a framework that explicitly separates these two responsibilities. A deterministic, system-agnostic stage first extracts compact yet faithful multimodal evidence from raw telemetry to formulate a set of evidence cards, while an LLM-based stage diagnoses the root cause solely based on this structured evidence through a restricted, read-only interface, without reading raw telemetry or executing code. In EviRCA, all system-specific knowledge is encapsulated in a lightweight declarative adapter, enabling the same reasoning engine to operate across heterogeneous systems without architectural changes.

This work makes the following contributions:

- **We identify the principal bottleneck of goal-driven LLM-based RCA.** Through process-level failure analysis and controlled ablations, we show that the dominant challenge is reasoning over surfaced evidence rather than searching raw telemetry with LLMs. This provides a new understanding of how LLMs should be incorporated into telemetry-grounded diagnosis.
- **We propose EviRCA, a two-stage framework for goal-driven RCA.** It separates deterministic evidence extraction from LLM-based semantic reasoning, represents heterogeneous telemetry as compact multimodal evidence cards, and isolates system-specific knowledge through a

declarative adapter so that the same engine generalizes across multiple enterprise systems.

- **We perform extensive experiments on the OpenRCA benchmark.** Across three enterprise systems and two LLMs from different model families, EviRCA improves the Correct rate to 40.6–43.9% compared with at most 15.2% for the baselines with matched models, while reducing token consumption by 15–26 $\times$. Specifically, it successfully solves hard-tier cases requiring recovery of the complete tuple including time, component and reason, which is a setting in which previous methods report 0% accuracy.

II. BACKGROUND AND MOTIVATION

A. Goal-Driven Root-Cause Analysis

OpenRCA frames RCA as a goal-driven task over telemetry [14]. Each case consists of a 30-minute window of telemetry and a natural-language query describing a failure that occurred within it. The telemetry is multimodal. Metrics are per-component time series of resource usage (CPU, memory, disk, network) together with application-level indicators such as request rate and latency. Traces record the call graph between components, along with each call’s latency and, where recorded, its success or failure. Logs, when available, are textual event streams that can surface failures metrics miss, such as a burst of garbage-collection messages. A method must therefore fuse evidence across these modalities, because a fault invisible in one is often conspicuous in another.

The query asks for a subset of the root cause’s three attributes (time, component, and reason); a window may contain more than one fault. Scoring is exact and all-or-nothing: components and reasons must belong to a closed, system-specific candidate set, and a case is judged Correct only when every queried attribute matches the ground truth (the precise metric is defined in Section IV-A). This contract rewards a single precise decision over broad exploration, which is what makes the task challenging.

B. Structural Heterogeneity Across Systems

The three systems share the task but differ in nearly every structural respect (Table I). First, they differ in their component hierarchy: Bank exposes a flat set of pods, whereas Telecom and Market are organized into node, pod, and service levels. Consequently, the level at which a fault must be reported and the manner in which a symptom propagates between levels vary from system to system. Second, they differ in the modalities that are actually present: Telecom lacks logs, and only Market exposes a service mesh, so the evidence available for fault attribution is not uniform. Third, they differ in time encoding; timestamps are recorded in units that range from seconds to milliseconds across files, making raw comparison impossible without normalization. Finally, they differ in how faults manifest and in how many possible reasons they admit. For example, a failure may surface as a database on/off flag on Telecom, as garbage-collection logs on Bank, or as mesh-edge status codes on Market. The number of distinct reasons

TABLE I
THE THREE OPENRCA SYSTEMS DIFFER ON EVERY STRUCTURAL AXIS, SO A METHOD MUST ABSORB THESE DIFFERENCES.

	Bank	Telecom	Market
Cases	136	51	148
Candidate levels	pod	node, pod, service	node, pod, service
Candidate components	14 pod (6 service)	22 node, 8 pod, 13 service	6 node, 40 pod, 10 service
Vertical topology	none	node-pod co-location	encoded in pod names; failover
Modalities	metric, trace, log	metric, trace (no log)	metric, trace, log
Golden signal	synthetic service tests	single gateway (OSB)	per service
Reason types	8	5	15
Time unit	metric s, trace ms, log s	all ms	metric s, trace ms, log s

also varies: 5, 8, and 15 for the three systems, respectively. These differences are structural, not cosmetic. A method that hard-codes the assumptions of one system will not transfer; therefore, handling all three systems simultaneously is the generalization challenge posed by this benchmark.

C. Agentic RCA and Its Costs

OpenRCA includes a reference method, RCA-Agent, that gives the LLM broad latitude to explore the telemetry: a planner repeatedly instructs an executor, which writes and runs Python over the raw data in a ReAct loop [15], inspecting and aggregating telemetry until it produces an answer. This reflects a broader class of autonomous RCA agents that operate on raw or lightly structured telemetry through tools or generated code [18], [19], [20], [21]. Such open-ended exploration is flexible, but it requires the model to process large volumes of raw telemetry across many steps per case, making it costly and unstable—as we quantify in Section IV-C and dissect in Section IV-E. EviRCA departs from this paradigm. Instead of letting the model explore raw telemetry freely, it enforces a strict separation: a deterministic stage handles the detection and extraction that can be done reliably without an LLM, while the LLM is reserved for the final reasoning step that truly requires it.

III. APPROACH DESIGN

A. Overview

Goal-driven root-cause analysis over a microservice system decomposes into three sequential subtasks: 1) detecting anomalies in multi-modal telemetry, 2) forming candidate localizations, and 3) discriminating the true root cause and its reason from competing candidates. Prior studies, such as RCA-Agent, collapse all three subtasks into one agentic loop: an LLM repeatedly writes and runs Python over the raw telemetry until it commits an answer. EviRCA instead matches each subtask to the mechanism suited to it (Fig. 1). Because detection and evidence extraction are deterministic, repeatable signal-processing tasks, EviRCA assigns them to a system-agnostic stage that reduces raw telemetry to faithful, compact evidence. Discrimination and attribution require semantic judgment over correlated candidates, so EviRCA assigns them to an LLM stage that reasons over the extracted evidence through a small set of predefined, read-only tools.

As shown in Fig. 1, EviRCA takes two inputs: the multimodal telemetry recorded within the query window, and the natural-language query that names the target attributes to recover. It then proceeds through two stages. The first stage, Deterministic Evidence Extraction (Sec. III-B), runs entirely in code and reduces the raw telemetry to a compact set of structured *evidence cards*, each describing one anomalous (component, resource-type) pair. The second stage, Reasoning over Structured Evidence (Sec. III-C), hands an LLM these cards and lets it discriminate the root cause through read-only tools, without ever reading raw telemetry or executing code. Binding the two stages together is a Declarative Adapter (Sec. III-D), a per-system specification of objective structure such as the telemetry layout, the candidate hierarchy, and the deployment topology, which both stages consult so that the engine itself contains no per-system logic. Finally, EviRCA outputs a $\langle \text{time, component, reason} \rangle$ prediction for each fault, reconciled to the number of injected faults the case contains.

The following subsections describe the two stages and the adapter in turn.

B. Deterministic Evidence Extraction

The purpose of the first stage is to provide the reasoning stage with something small, faithful, and complete to work with, in place of raw telemetry. Its role in the pipeline is to perform the two subtasks that require no language model, anomaly detection and candidate localization, and to do so deterministically. Doing this work in code rather than in the LLM brings three benefits that an agentic loop lacks. The output is reproducible, since the same window always yields the same cards. The model can no longer hallucinate data points or schema fields, since it never sees the raw series. Finally, the evidence it passes downstream is already compact, which keeps the reasoning stage cheap.

The stage takes the raw telemetry in the query window, together with the structure declared by the adapter, and produces a set of evidence cards. Each card records one anomalous (component, resource-type) pair: its onset time, the KPIs that support it, and a set of *coarse* reason candidates. Before any window is cut, EviRCA computes a robust statistical baseline for every component KPI over the full day, so that a short fault inside the window is judged against the component’s own normal behavior. The stage then runs four steps: shape-

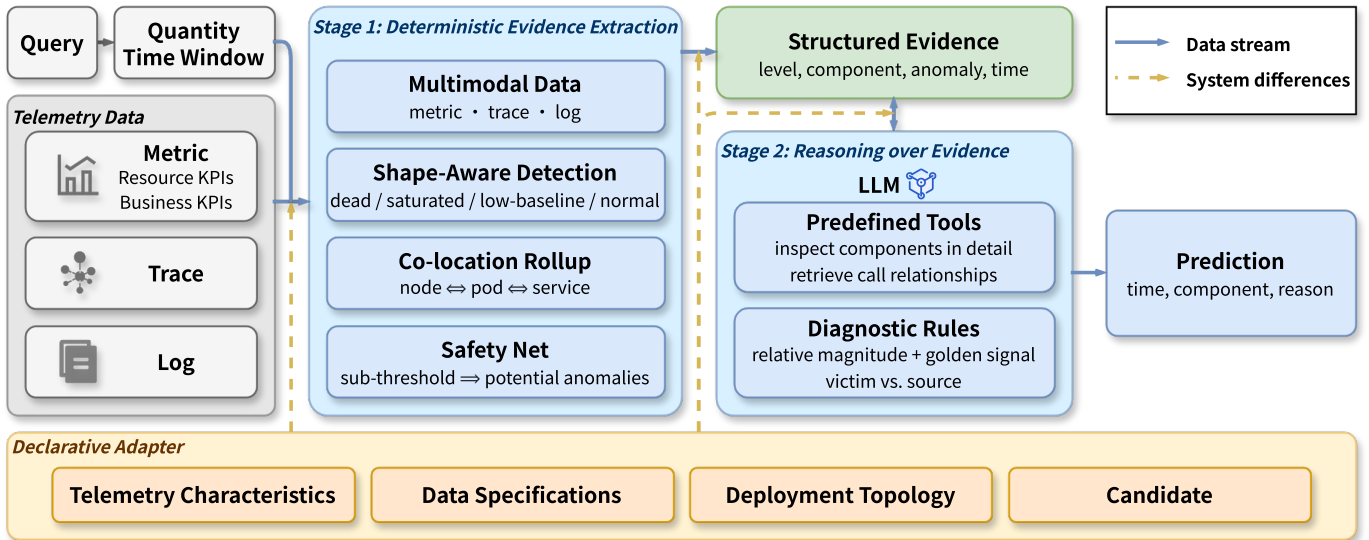


Fig. 1. Overview of EviRCA. Stage 1 reduces raw multi-modal telemetry to structured evidence cards through deterministic signal processing, and Stage 2 lets an LLM reason over them to predict the $\langle$ time, component, reason $\rangle$ of each fault. A Declarative Adapter supplies per-system specifications to both stages, keeping the engine system-agnostic.

aware detection finds the anomalous series, multimodal card construction normalizes anomalies from every modality into the common card format, co-location rollup re-levels evidence to the granularity at which a fault must be reported, and a sub-threshold fallback guarantees that the output is never empty. We describe each step in turn.

Shape-aware detection. This step takes each component KPI’s full-day series and its baseline and decides whether, and when, the series becomes anomalous inside the window. A single anomaly test cannot serve every KPI, because operational telemetry contains series of very different shapes, and a test tuned for one shape misfires on another. EviRCA therefore first classifies each series by the shape of its daily distribution and then applies a detector matched to that shape. We distinguish four shapes, each with its characteristic signature and a correspondingly specialized detector:

- *Dead* series have effectively no variance and carry no signal, so they are discarded.
- *Saturated* series sit persistently at the top of their range, with even their low percentiles close to the maximum, so an absolute upper threshold can never fire; such series are flagged and routed to the fallback, since their fault is invisible to a metric threshold and must be recovered through other evidence.
- *Low-baseline* series rest near zero and move only in sparse spikes, so a purely relative test reads ordinary noise as a large excursion; they must therefore clear an absolute-magnitude floor on top of the relative criterion, which keeps a jump from zero to a negligible value from being reported as a fault.
- *Normal* series are well behaved and admit a standard test for a sustained, significant excursion beyond a robust band around the baseline.

Across all shapes, an anomaly is accepted only when the excursion persists for several consecutive samples and is large relative to the band, which suppresses isolated spikes and marginal breaches. Treating all four shapes alike would miss faults on the dead and saturated series and fabricate them on the low-baseline ones.

Multimodal card construction. This step takes the anomalies surfaced from every available modality and normalizes them into one card format. EviRCA runs a detector per modality: the shape-aware test above over metrics, a trace detector that flags failed or abnormally slow calls, and a log detector that flags telltale bursts such as a surge of garbage-collection messages signalling memory pressure. Each detected anomaly becomes a card for its (component, resource-type) pair, annotated with coarse reason candidates drawn from the adapter’s resource-type-to-reason map. When a resource type is consistent with more than one reason, the card carries several candidates, and the choice among them is deferred to the reasoning stage. Running every modality matters because a fault that is silent in one is often loud in another: a memory exhaustion that never trips a metric threshold still appears as a garbage-collection burst in the logs, and a node fault with no node-level metric still appears as latency in the traces. Extracting all modalities is what makes the card set high-recall, so that the reasoning stage rarely has to search for evidence the extraction stage failed to surface.

Co-location rollup. This step takes the cards produced so far, together with the deployment topology declared by the adapter, and adjusts the granularity at which each fault is reported. It is needed because a fault sometimes leaves no signature on the component that actually carries it. A node fault, for instance, may surface only as latency on the pods the node hosts, never on a node-level metric, so the raw cards point

TABLE II
TOOLS AVAILABLE TO THE LLM REASONING STAGE.

Tool	Input	Output	Description
<code>list_entries</code>	—	Detected entries (component, type, onset, KPIs); weak leads if none	Starting leads from the deterministic layer
<code>get_component</code>	component	Its entries, blind/saturated types, topology, available KPIs	Full evidence for one component
<code>get_kpi_series</code>	component, KPI	Downsampled in-window series with day baseline	Confirm a signal’s shape and anomaly
<code>get_golden_signal</code>	—	Business-KPI anomalies and their onset	System failure timing, not location; fallback
<code>get_call_graph</code>	—	Services ranked by latency / fail rate; call edges	Localize a network fault; trace propagation
<code>get_log_summary</code>	component	Per-log-name frequency vs. day baseline	Catch metric-blind faults (e.g. a GC burst)
<code>get_log_sample</code>	component, log name	Sampled raw log lines	Read log content to disambiguate a reason
<code>submit_answer</code>	root causes	Acceptance or a correctable error	Commit and validate the final answer

at the victims rather than the source. EviRCA rolls evidence up in two directions. Vertically, it groups the affected pods by their host node: when several co-located pods on one node degrade together, it emits a card for their shared node, whereas a single pod degrading while its co-located neighbors stay healthy keeps its card at the pod level. Horizontally, it groups the affected pods by their service: when the pods of one service show the same anomaly, it emits a card for the service and removes the subsumed pod cards. The direction and grouping are driven entirely by the declared topology, so this primitive applies to any system that declares one. This step recovers level-spanning faults that would otherwise be reported at the wrong level, and it does so without discarding the underlying pod evidence, which stays available to the reasoning stage.

Sub-threshold fallback. The detector above is deliberately high-precision, so in a few cases it surfaces no card at all. This step guarantees that the reasoning stage is never handed an empty set: when no series crosses the detection threshold, EviRCA emits the closest sub-threshold leads, ranked by how far they deviate from their baseline. These leads are kept separate from the high-precision cards and used only as a last resort, so they form a safety net without diluting the evidence on the cases that do produce cards.

Together, the four steps turn a window of raw, heterogeneous telemetry into a card set that is faithful, modality-complete, reported at the right granularity, and never empty, which is exactly the input the reasoning stage needs.

C. Reasoning over Structured Evidence

The second stage performs the one subtask that genuinely calls for an LLM: discriminating the true root cause and its reason from the competing candidates the first stage surfaced. This is hard not because the evidence is missing but because it is correlated. The same fault leaves traces on many components at once, so the cards alone do not say which component is the cause, which is a downstream victim, and which of several plausible reasons actually holds. Resolving this calls for semantic judgment, which is where an LLM is uniquely useful, but EviRCA confines that judgment to a bounded, auditable interaction rather than open-ended exploration.

The stage takes the evidence cards, the query, and a small set of predefined, read-only tools, and it returns a (time,

component, reason) tuple for each fault. The LLM works only through these tools, and never reads raw telemetry or runs code. It inspects the cards and their supporting signals as needed, settles on the root-cause component and reason, and submits its answer, which the system then reconciles to the number of injected faults. The answer time is taken from the supporting card’s onset rather than computed by the model, which removes a frequent source of timing error.

The tools are capability-gated: each is offered only when the system actually has the modality behind it, so the call-graph tool appears only when traces exist and the log tools only when logs do. Through these tools, summarized in Table II, the model inspects the cards and their supporting signals and commits an answer. A *golden signal* here is a business-level KPI that reflects user-facing health, such as aggregate request rate, success rate, or mean response time. It tells the model *when* the system degraded, which separates a real fault from incidental noise, but not *where* the fault is.

These tools let the model reason from the evidence rather than follow a fixed script, and three principles guide that reasoning. It ranks candidates by relative magnitude rather than mere presence, because a single fault triggers anomalies in many correlated KPIs and only the relative size of a specific signature separates the cause from its effects. It follows the call graph from the loudest victim back to the most upstream abnormal component, since the loudest signal is usually a symptom rather than the source. And, when a golden signal exists, it uses the signal’s onset to reject leads whose timing does not line up. Because the action space is bounded and the evidence is already reduced, this stage costs far fewer tokens than an open-ended code loop and gives a stable answer across runs.

D. Declarative Adapter

All system-specific knowledge in EviRCA resides in one declarative adapter, and the shared engine has no per-system branches. An adapter declares only objective structure, never parameters fitted to the answers. It names the telemetry files, their columns, and their units; the per-modality specifications that drive the detectors; the deployment topology used by the co-location rollout; the candidate hierarchy and its components; the mapping from resource type to coarse reason candidates;

and an optional short paragraph of structural guidance. Writing an adapter is a modest, one-time effort per system, and it is the only manual step needed to configure a new system. Because both stages read their structure from the adapter, the same engine runs all three benchmark systems unchanged, despite their differences in topology, modality coverage, and telemetry encoding.

E. Running Example

Figure 2 walks one Telecom failure through both stages. The query asks only for the root-cause reason of a single failure between 04:00 and 04:30.

The first stage reduces the window to six evidence cards from three modalities: two database services (`db_010`, `db_012`) with brief connection spikes around 04:01–04:03; two pods (`docker_004`, `docker_006`) flagged by the trace modality as isolated slow pods; and two nodes (`os_018`, `os_008`) with elevated CPU. The true cause, `docker_006`, surfaces through the trace modality, and the rollup keeps it at the pod level because its co-located neighbor `docker_002` stays healthy.

The second stage then discriminates. The golden signal deviates at 04:19, so the model discards the database spikes, which are transient and far too early. The trace ranks `docker_006` highest in latency, at a $3.9\times$ ratio. Its container CPU stays above the 99th percentile from 04:19 onward while its co-located neighbor shows nothing. A node-wide CPU fault would have hit both pods, so the elevated node CPU on `os_018` is collateral, not the cause. Inspecting `docker_006`’s downstream database rules out propagation from below. The model outputs `docker_006`, CPU fault. The first stage put the right evidence on the table, and the second stage chose correctly among the distractors.

IV. EVALUATION

The evaluation is framed around the following four core Research Questions (RQs): **RQ1 (Effectiveness)**: To what extent does EviRCA outperform state-of-the-art baselines in root-cause localization across structurally heterogeneous microservice systems and diverse LLM backends? **RQ2 (Efficiency and Operational Overhead)**: How does EviRCA compare to existing agentic workflows in terms of computational efficiency and wall-clock execution latency? **RQ3 (Ablation Study)**: What are the individual contributions of EviRCA’s core architectural components, and to what degree are the performance gains decoupled from system-specific prompt engineering? **RQ4 (Error Analysis)**: What are the primary taxonomies and root causes of EviRCA’s residual failures, and what structural bottlenecks do they reveal?

A. Experimental Setup

a) *Benchmark and systems*: OpenRCA [14] comprises three enterprise microservice systems that differ sharply in topology, modality coverage, and telemetry encoding. We report them in a fixed order: *Bank* (136 cases), *Telecom* (51), and *Market* (148). We treat Market as one system, the union of

its two deployments `cloudbed-1` and `cloudbed-2`. Each case is a natural-language query over a 30-minute window that asks for some subset of a root cause’s $\langle \text{time, component, reason} \rangle$.

b) *Evaluation Metric*: We use OpenRCA’s scorer. A case is *Correct* only if every queried field matches the ground truth and the number of predicted root causes equals the number of injected faults. Components and reasons must come from the system’s closed candidate sets, and times must fall within ± 60 s. This all-or-nothing *Correct* rate is our primary metric. *Partial* credits the fraction of queried fields a case answers correctly. Difficulty follows the query templates: *easy* (one queried element, task 1–3), *middle* (two, task 4–6), and *hard* (all three, task 7).

c) *Baselines*: We compare against the two methods reported for OpenRCA [14], run with the same two LLMs. *RCA-Agent* is the benchmark’s controller–executor agent: a planner iteratively instructs an executor that writes and runs Python over the raw telemetry in a stateful kernel. *Oracle Sampling* answers from a sampled slice of telemetry in a single LLM call. It serves as an idealized upper bound for sampling-based methods, as it draws exclusively from KPIs pre-selected by domain engineers to be diagnostic—reducing the KPI space by 95% while retaining the most causally relevant signals.

d) *LLM backends*: This work adopts two LLMs in different model families, including `deepseek-v4-pro` and `qwen3.7-plus`, to comprehensively test EviRCA and the comparison methods.

B. RQ1: Accuracy

Table III presents the experimental results among EviRCA and the baselines on OpenRCA. It demonstrates that EviRCA outperforms both baselines across systems, LLM backends and evaluation metrics. Specifically, EviRCA achieves a weighted overall *Correct* rate of 43.9% when paired with `deepseek-v4-pro`, and 40.6% with `qwen3.7-plus`. In stark contrast, the top-performing baseline configuration yields a maximum *Correct* rate of only 15.2%, establishing a substantial performance gap where EviRCA delivers a near threefold improvement in overall accuracy. This performance margin is particularly pronounced in the Bank and Telecom systems. For instance, on the Telecom dataset under identical model backends, EviRCA advances the *Correct* score to 62.7%, whereas RCA-Agent and Oracle Sampling underperform at 11.8% and 19.6%, respectively. The same ordering holds on the Partial metric: EviRCA recovers 54–56% of queried fields overall against at most 27% for any baseline, so it gets more of the answer right even on cases it does not fully solve.

To test whether knowing *where* to look is the binding constraint, we compare EviRCA against the Oracle Sampling baseline, which benefits from privileged information. While that baseline method has prior access to a curated list of KPIs, EviRCA achieves an accuracy rate two to four times higher. The end-to-end task comprises multiple sub-processes, and simply knowing *where* to start detection is insufficient to bridge this gap. The challenge lies not only in locating

Query: a single failure occurred between 04:00 and 04:30. What is the root-cause reason?

Stage 1 - Deterministic evidence extraction

Raw telemetry (portion)

docker_006 - container_cpu_used



sustained 37-40 (over p99) from 04:19

os_018 - CPU_util_pct (node)



rises 04:17, peaks 8.3, returns by 04:22

db_010 - Session_pct (service)



transient spike to 1.6 at 04:03

4:00

4:30

Evidence cards (Stage 1 output)

docker_006 - pod - trace_latency onset 04:17
⇒ candidate: **CPU fault**

docker_004 - pod - trace_latency onset 04:17
⇒ candidate: **CPU fault**

os_018 - node - metric onset 04:22
⇒ candidate: **CPU fault**

os_008 - node - metric onset 04:27
⇒ candidate: **CPU fault**

db_010 - service - metric onset 04:01
⇒ candidate: **db connection limit**

db_012 - service - metric onset 04:00
⇒ candidate: **db connection limit**

Stage 2 - LLM reasoning over evidence

- 1 get_golden_signal
Business KPI avg_time breaks at 04:19
⇒ drop **db_010** and **db_012** (early, transient)
- 2 get_call_graph
docker_006 has the highest latency, 3.9x
⇒ focus on **docker_006**
- 3 get_component(**docker_002**)
Co-located neighbor is healthy
⇒ the fault is pod-level; the node CPU on **os_018** is collateral, not the cause
- 4 get_component(**db_003**)
Downstream database is clean
⇒ not propagated from below
- 5 submit_answer
⇒ **docker_006 - CPU fault - 04:17:00**
[check] matches ground truth

Fig. 2. A running example of EviRCA on the Telecom dataset: Stage 1 reduces raw multi-modal telemetry to evidence cards, and Stage 2 reasons over them to pick the true root cause among the distractors.

TABLE III

ROOT-CAUSE ACCURACY ON OPENRCA (%). *Correct* REQUIRES ALL QUERIED ATTRIBUTES TO MATCH; *Partial* CREDITS PARTIALLY CORRECT PREDICTIONS. *Overall* IS WEIGHTED BY CASE COUNT (BANK 136, TELECOM 51, MARKET 148; 335 TOTAL). BEST RESULTS IN **BOLD**.

Method	LLM	Bank		Telecom		Market		Overall	
		Correct	Partial	Correct	Partial	Correct	Partial	Correct	Partial
Oracle Sampling	deepseek-v4-pro	12.5	20.7	19.6	25.1	14.9	22.3	14.6	22.1
	qwen3.7-plus	9.6	17.6	29.4	36.6	15.5	32.4	15.2	27.0
RCA-Agent	deepseek-v4-pro	17.6	23.1	11.8	19.6	9.5	19.1	13.1	20.8
	qwen3.7-plus	22.1	29.1	9.8	16.0	6.8	16.3	13.4	21.5
EviRCA	deepseek-v4-pro	52.2	63.2	62.7	65.4	29.7	46.7	43.9	56.2
	qwen3.7-plus	47.8	60.8	56.9	60.8	28.4	46.2	40.6	54.4

relevant telemetry data but also in identifying the fault and interpreting the root cause amidst competing evidence, which will be further discussed in Section IV-E.

Figure 3 breaks accuracy down by system and query difficulty. While accuracy inherently scales down as difficulty increases from easy to hard, EviRCA consistently preserves a clear separation from the baselines at all tiers. This contrast peaks on the hardest tier, which demands the joint prediction of the full triple, i.e., the fault’s time, component, and reason together. On these hard cases, EviRCA successfully resolves 17.6%–29.4% of Bank incidents and 55.6% of Telecom incidents under both LLMs. In comparison, prior literature reports a 0% success rate on this hard tier across all evaluated models [14]. The two newer LLMs we evaluate do lift the baselines off that floor on a few cells: RCA-Agent reaches 11.8% on Bank and Oracle Sampling 11.1% on Telecom. Yet no baseline cell

exceeds 11.8%, whereas EviRCA’s weakest hard cell already stands at 17.6% and its strongest at 55.6%. Solving the hard tier therefore calls for the joint judgment that EviRCA’s design supplies, not merely a stronger model.

C. RQ2: Efficiency and Operational Overhead

Table IV reports the per-case operational cost evaluated under the deepseek-v4-pro backend. The empirical tracking reveals that EviRCA substantially reduces resource consumption, utilizing 15×–26× fewer tokens per incident compared to RCA-Agent. This drastic token economy stems directly from EviRCA’s architectural design: it executes a single, structured reasoning pass over pre-reduced, highly concentrated evidence, thereby completely bypassing the expensive, multi-turn loops of reasoning and code execution over raw, high-volume telemetry data.



Fig. 3. Strict (Correct) accuracy by system and query difficulty: three methods (columns) under two LLMs (rows). Within each panel, bars group by system and are shaded by difficulty.

TABLE IV
PER-CASE COST WITH DEEPSEEK-V4-PRO, WHERE *ratio* = RCA-AGENT / EVIRCA.

	Bank	Telecom	Market
EviRCA tokens/case	101.8K	48.3K	66.4K
RCA-Agent tokens/case	1.53M	1.24M	1.54M
<i>ratio</i>	15.0×	25.7×	23.3×
EviRCA time/case (s)	182.3	31.7	40.8
RCA-Agent time/case (s)	577	622	685
<i>ratio</i>	3.2×	19.6×	16.8×

Apart from token efficiency, EviRCA achieves a 3.2× to 19.6× speedup in per-case wall-clock execution time. We note that the latency reduction is relatively less pronounced within the Bank system. This stems from how the Bank system encodes traces: they carry no call status or explicit callee and must be reassembled from per-span parent links, over the largest trace volume of the three systems, so EviRCA reconstructs the relevant call graph separately for each case. Telecom and Market do not incur this overhead, since their traces can be summarized once upfront, into a compact per-minute call summary that every case reuses, leaving per-case trace work negligible.

Overall, EviRCA’s efficiency advantage holds across all three systems. The token reduction, the dominant and model-independent cost driver, remains at an order of magnitude or more (15–26×) on every system, while wall-clock time drops by 3.2–19.6×. Even on Bank, where per-case trace reconstruction narrows the time margin, EviRCA remains both substantially cheaper and faster than the agentic baseline.

D. RQ3: Ablation Study

To investigate the individual contributions of EviRCA’s core components, we conduct a leave-one-out ablation study

TABLE V
THE IMPACT ON THE *Correct* rate (IN PERCENTAGE POINTS) VIA LEAVE-ONE-OUT MODULE REMOVAL, WHICH IS EVALUATED WITH DEEPSEEK-V4-PRO.

Module removed	Bank	Telecom	Market
Trace	−3.7	−45.1	−0.7
Co-location rollup	−	−37.3	−1.4
Per-system guidance	+0.7	−11.8	0.0
Golden signal	0.0	−3.9	0.0
Safety net	−2.9	0.0	−3.4
Log modality	−1.5	−	−

using the deepseek-v4-pro backend. Table V reports the deviation in the *Correct* rate when each module is disabled. Crucially, the performance impact of each component is highly system-specific, closely aligning with the underlying telemetry characteristics and fault distributions of each target system.

On the Telecom system, disabling either the trace modality or the co-location rollup leads to a catastrophic performance drop, i.e., −45.1 and −37.3 percentage points in the *Correct* rate, respectively. These two primitives enable EviRCA to localize Telecom’s node faults, which leave little or no signature on the node’s metric: such a fault surfaces as latency on the node’s co-located pods through the traces, and the co-location rollup is what aggregates those slow pods back to the responsible node. Without either, the node fault cannot be reported at its correct level. On Bank, by contrast, no single module dominates: the trace modality, the safety net, and the log modality each move the *Correct* rate by only a few points (−3.7, −2.9, and −1.5). This reflects a system whose fault signatures are spread across resource types and modalities rather than concentrated in one primitive.

The per-system guidance consists of a short hand-written paragraph, and it is the only point where expert knowledge about a system enters the pipeline. We call attention to it

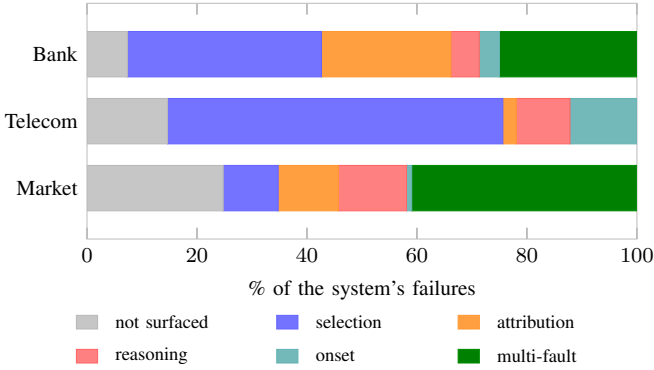


Fig. 4. Failure profile by the pipeline stage at which each failure occurs, deepseek-v4-pro and qwen3.7-plus. *not surfaced* = the true component never reached the evidence (a recall gap); *selection/attribution/reasoning* are wrong decisions over *already-surfaced* evidence; *multi-fault* cases are broken out separately.

TABLE VI
DISTRIBUTION AND OVERLAP OF FAILURE CASES BETWEEN DEEPSEEK-V4-PRO AND QWEN3.7-PLUS, WHERE *shared* REPRESENTS THE PERCENTAGE OF CASES FAILED BY BOTH MODELS OUT OF ALL CASES FAILED BY AT LEAST ONE MODEL.

System	both-fail	one model only	both-pass	shared
Bank	61	14	61	81.3%
Telecom	19	3	29	86.4%
Market	97	16	35	85.8%

because, if EviRCA’s advantage stemmed from prompt tuning to the benchmark, removing this guidance should cause accuracy to collapse. It does not: removing the guidance impairs performance only on Telecom (-11.8 points, where propagation reasoning matters most); on Market it has no effect (0.0), and on Bank it even yields a slight improvement (+0.7 points). These gains are therefore architectural, not an artifact of prompt tuning. Finally, as discussed in Section IV-E, Market’s bottleneck lies upstream of its modules; accordingly, no single ablation alters its Correct rate by more than three percentage points.

E. RQ4: Failure Analysis

We categorize each failure case by both its specific breakdown stage within the pipeline (Fig. 4) and its cross-model consistency (Table VI). This fine-grained categorization yields three key insights that refine our core empirical findings.

Most residual failures stem from erroneous decisions over surfaced evidence rather than a lack of coverage. Among single-fault failures, the extraction stage fails to surface the true component in only about a quarter of cases (input-missing 22.5%). The rest occur on evidence that *was* surfaced: a downstream component with a stronger anomaly chosen over the true root cause (35%), the wrong reason among the candidates (21%), or an incorrect propagation chain (14%). This matches the privileged Oracle Sampling result from RQ1: being given where to look does not settle which candidate is the cause.

Failures are largely shared across models, which explains the narrow spread between LLMs. 81–87% of failing cases fail under *both* models (Table VI), and only 13–19% are model-specific. The residual error is dominated by genuinely ambiguous decisions, such as network delay versus packet loss, or a propagated symptom versus its source, that swapping models within this capability tier does not resolve. This explains why the two LLMs perform so similarly in Table III: within the same capability tier, a stronger model cannot resolve the ambiguity that the evidence inherently leaves.

The systems have distinct failure profiles (Fig. 4). On Telecom, failures are almost entirely selection errors (61%): the correct node is usually surfaced in the evidence as a low-confidence lead, but a competing lead is chosen instead. Bank splits across selection, attribution, and multi-fault. Market is dominated by *multi-fault* (41%) and by *un-surfaced* evidence (25%), and that un-surfaced evidence is concentrated in network faults. Among Market’s failing network-reason cases, the true component never reaches the evidence in 89.6% of them, versus 8.5% on Bank and 0% on Telecom. This is a coverage limit of the evidence layer for network signals rather than a reasoning failure, and we return to it among the limitations.

F. The Per-System Picture

Reading the four questions by system, rather than by question, clarifies the overall picture. On Bank, EviRCA is accurate (52.2%) but its modules are individually small (RQ3), and its failures split across selection, attribution, and multi-fault (RQ4), so the bottleneck is judgment rather than coverage. On Telecom, EviRCA is strongest (62.7% Correct) and most clearly model-driven: trace and the co-location rollout are decisive (RQ3), and the residual error is almost entirely selection among surfaced leads (RQ4). On Market, EviRCA is weakest (29.7%), and its modules are nearly inert (RQ3) because the binding limits lie upstream: a large multi-fault share, and network faults that the evidence layer does not surface (RQ4). The same engine therefore succeeds or struggles for system-specific reasons, and those reasons are coverage and ambiguity rather than a failure of the reasoner.

V. DISCUSSION AND LIMITATIONS

A. Further Discussions

The results show that, for microservice RCA, the key issue is to draw a proper boundary between deterministic evidence extraction and LLM reasoning. EviRCA assigns anomaly detection and candidate localization to a deterministic stage. It then lets the LLM infer the root cause from structured evidence cards. This constrained design improves the overall Correct rate to 40.6%-43.9%, while the best baseline reaches only 15.2%. It also uses 15-26 times fewer tokens and reduces per-case latency by 3.2-19.6 times. These results suggest that free-form exploration over raw telemetry is not necessary in this setting. It can also introduce context expansion, schema hallucination, and run-to-run instability. The failure analysis further shows that most residual errors come from incorrect judgments over extracted evidence. Among single-fault

failures, the true component is missing from the extracted evidence in only 22.5% of cases. For example, the model may select a downstream victim with a stronger anomaly as the root cause, choose the wrong reason among similar candidates, or misinterpret the propagation chain. In addition, 81%-87% of failures are shared by the two LLMs. This suggests that the remaining difficulty is not easily solved by replacing the model within the same capability tier. Instead, the main challenge is to make the correct interpretation among correlated and interfering evidence.

B. Limitations

The main limitation of EviRCA is that the evidence layer determines its performance ceiling. If a true fault signal is not included in the evidence cards due to missing modalities, coarse granularity, or unsuitable detection rules, the downstream LLM cannot recover it. In failing network-reason cases, the true component is absent from the evidence in 89.6% of Market cases, compared with 8.5% in Bank and 0% in Telecom. This helps explain why Market has lower accuracy than the other two systems. Future work may address this issue by adding richer modalities, using finer-grained features. In addition, EviRCA inherits several assumptions from OpenRCA, such as closed sets of candidate components and reasons, known fault counts, and available topology information. These assumptions may not always hold in real production environments.

VI. RELATED WORK

A. Root Cause Analysis for Microservices

As discussed previously, root-cause analysis over microservice telemetry is commonly decomposed into anomaly detection, localization of faulty components, and attribution of failure time, affected component, and underlying reason. Prior to recent LLM-based work, microservice RCA primarily relied on localization methods grounded in distributed traces, metric time series, and service dependency graphs. Representative approaches include random-walk ranking over service call graphs [22], causal-graph construction [23], [24], trace analysis [25], causal inference and discovery [26], [27], [28], deep-learning-based QoS debugging [29], and multimodal localization [30], [31]. Although effective, these methods typically require labeled service topology, supervised training data, or deployment-specific model calibration, limiting their transferability across systems.

B. LLM-based RCA Agents

Recent work has applied LLMs to RCA along two lines. One line operates on textual incident artifacts rather than raw telemetry, producing incident summaries, coarse root-cause categories, or mitigation recommendations from historical reports [11], [12], [13]; these outputs support triage but do not localize faults within live telemetry. Another line gives the model direct control over telemetry analysis. Tool-augmented and code-generating agents retrieve and analyze telemetry autonomously [18], [19], [32], with later systems adding

SOP-guided multi-agent orchestration [20], graph-augmented workflows [33], multimodal tool invocation [21], and meta-level causal knowledge [34]. RCA-Agent is representative of this direction, allowing an LLM to iteratively write and execute Python over raw telemetry in a ReAct loop [14], [15]. These approaches therefore place substantial autonomy in the model: the LLM chooses what evidence to inspect and often generates code to inspect it. A smaller line of work pre-structures telemetry before model invocation [35], [36], but still processes the resulting evidence through autonomous multi-agent loops.

C. LLM Agents for Software Engineering

Beyond RCA, LLM agents have advanced code generation, issue resolution, and test generation through benchmarks such as SWE-bench [4] and systems such as SWE-agent [5], OpenHands [6], and MetaGPT [7]. This progress relies on natural-language-friendly inputs and inexpensive test-based verification, which turn open-ended exploration into a checkable loop. Even in that setting, constrained action spaces often outperform broader autonomy, as shown by structured code search [37], localize-then-repair pipelines without autonomous control flow [38], and constrained agent interfaces [5]. Telemetry-grounded RCA lacks both conditions: its inputs are heterogeneous numeric signals, and predicted root causes have no cheap executable check. Code generation over raw telemetry therefore inherits the cost of open-ended exploration without the test oracle that makes such exploration effective in software-engineering tasks.

VII. CONCLUSION

We presented EviRCA, a framework that addresses the mismatch between LLMs and raw microservice telemetry by decoupling deterministic evidence extraction from LLM-based reasoning. A system-agnostic extraction stage reduces heterogeneous metrics, traces, and logs into compact, faithful multimodal evidence cards, with system-specific structure isolated in a lightweight declarative adapter. The LLM then diagnoses the root cause only through a restricted, read-only interface over these cards, without accessing raw telemetry or executing code. Across three enterprise systems and two LLMs, EviRCA achieves a Correct rate of 40.6–43.9%, compared with at most 15.2% for the strongest OpenRCA baseline under matched models. It also reduces token consumption by 15–26 $\times$ and per-case latency by 3.2–19.6 $\times$, while solving hard-tier cases requiring the complete time, component, and reason tuple. Through ablation and process-level failure analysis, we show that the main bottleneck is not open-ended search over raw telemetry, but judgment over evidence that has already been surfaced. At the same time, cases such as Market network faults show that the evidence layer still determines the performance ceiling when key signals are missing. These findings make the extraction–reasoning boundary the central design lever for LLM-based RCA, and point to richer, finer-grained, and adaptive evidence extraction as the main route to further progress.

VIII. DATA AVAILABILITY

To support reproducibility, our implementation, experiment scripts, and a README with reproduction instructions are publicly available at <https://github.com/yuhao541/EviRCA>. Experiments are performed with the publicly available OpenRCA benchmark [14]; instructions for obtaining it are included in the repository.

REFERENCES

- [1] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, *Microservices: Yesterday, Today, and Tomorrow*. Cham: Springer International Publishing, 2017, pp. 195–216. [Online]. Available: https://doi.org/10.1007/978-3-319-67425-4_12
- [2] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey,” *ACM Comput. Surv.*, vol. 55, no. 3, Feb. 2022. [Online]. Available: <https://doi.org/10.1145/3501297>
- [3] N. Fu, G. Cheng, Y. Teng, G. Dai, S. Yu, and Z. Chen, “Intelligent root cause localization in microservice systems: A survey and new perspectives,” *ACM Comput. Surv.*, vol. 57, no. 12, Jul. 2025. [Online]. Available: <https://doi.org/10.1145/3736755>
- [4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQm66>
- [5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: agent-computer interfaces enable automated software engineering,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS ’24. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [6] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “Openhands: An open platform for AI software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=OJd3ayDDoF>
- [7] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [8] GitHub, “GitHub Copilot,” <https://github.com/features/copilot>, 2026, accessed: 2026-06-20.
- [9] OpenAI, “OpenAI Codex,” <https://openai.com/codex/>, 2026, accessed: 2026-06-20.
- [10] Anthropic, “Claude Code,” <https://claude.com/product/claude-code>, 2026, accessed: 2026-06-20.
- [11] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, “Recommending root-cause and mitigation steps for cloud incidents using large language models,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1737–1749.
- [12] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, “Automatic root cause analysis via large language models for cloud incidents,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 674–688. [Online]. Available: <https://doi.org/10.1145/3627703.3629553>
- [13] Y. Jiang, C. Zhang, S. He, Z. Yang, M. Ma, S. Qin, Y. Kang, Y. Dang, S. Rajmohan, Q. Lin, and D. Zhang, “Xpert: Empowering incident management with query recommendations via large language models,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639081>
- [14] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, “OpenRCA: Can large language models locate the root cause of software failures?” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=M4qNizQYpd>
- [15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_yluYUL-X
- [16] T. Kim, W. Park, H. Yun, and K. Lee, “Why do ai agents systematically fail at cloud root cause analysis?” 2026. [Online]. Available: <https://arxiv.org/abs/2602.09937>
- [17] E. Riddell, J. Riddell, G. Sun, M. Antkiewicz, and K. Czarnecki, “Stalled, biased, and confused: Uncovering reasoning failures in llms for cloud-based root cause analysis,” *ArXiv*, vol. abs/2601.22208, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285241191>
- [18] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, “Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, ser. CIKM ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 4966–4974. [Online]. Available: <https://doi.org/10.1145/3627673.3680016>
- [19] D. Roy, X. Zhang, R. Bhavne, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, “Exploring llm-based agents for root cause analysis,” in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 208–219. [Online]. Available: <https://doi.org/10.1145/3663529.3663841>
- [20] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei, “Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis,” in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 422–431. [Online]. Available: <https://doi.org/10.1145/3701716.3715225>
- [21] X. Zhang, Q. Wang, M. Li, Y. Yuan, M. Xiao, F. Zhuang, and D. Yu, “Tamo: fine-grained root cause analysis via tool-assisted llm agent with multi-modality observation data in cloud-native systems,” *IEEE Transactions on Services Computing*, vol. 18, no. 6, pp. 4221–4233, 2025.
- [22] M. Kim, R. Sumbaly, and S. Shah, “Root cause detection in a service-oriented architecture,” in *Proceedings of the ACM SIGMETRICS/International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS ’13. New York, NY, USA: Association for Computing Machinery, 2013, p. 93–104. [Online]. Available: <https://doi.org/10.1145/2465529.2465753>
- [23] J. Lin, P. Chen, and Z. Zheng, “Microscope: Pinpoint performance issues with causal graphs in micro-service environments,” in *Service-Oriented Computing: 16th International Conference, ICSOC 2018, Hangzhou, China, November 12-15, 2018, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2018, p. 3–20. [Online]. Available: https://doi.org/10.1007/978-3-030-03596-9_1
- [24] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “Microrca: Root cause localization of performance issues in microservices,” in *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–9.
- [25] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, and D. Pei, “Practical root cause localization for microservice systems via trace analysis,” in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, 2021, pp. 1–10.
- [26] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, “Causal inference-based root cause analysis for online service systems with intervention recognition,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 3230–3240. [Online]. Available: <https://doi.org/10.1145/3534678.3539041>
- [27] A. Ikram, S. Chakraborty, S. Mitra, S. K. Saini, S. Bagchi, and M. Kocaoglu, “Root cause analysis of failures in microservices through causal discovery,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.

- [28] S. Zhang, A. Fang, Y. Yang, R. Cheng, X. Tang, and P. He, "Dynacausal: Dynamic causality-aware root cause analysis for distributed microservices," 2025. [Online]. Available: <https://arxiv.org/abs/2510.22613>
- [29] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou, "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 19–33. [Online]. Available: <https://doi.org/10.1145/3297858.3304004>
- [30] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 553–565. [Online]. Available: <https://doi.org/10.1145/3611643.3616249>
- [31] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1750–1762.
- [32] B. Qiao, L. Li, X. Zhang, S. He, Y. Kang, C. Zhang, F. Yang, H. Dong, J. Zhang, L. Wang, M. Ma, P. Zhao, S. Qin, X. Qin, C. Du, Y. Xu, Q. Lin, S. Rajmohan, and D. Zhang, "Taskweaver: A code-first agent framework," 2024. [Online]. Available: <https://arxiv.org/abs/2311.17541>
- [33] Y. Tian, Y. Liu, Z. Chong, Z. Huang, and H.-A. Jacobsen, "Gala: Can graph-augmented large language model agentic workflows elevate root cause analysis?" 2025. [Online]. Available: <https://arxiv.org/abs/2508.12472>
- [34] S. Liang, P. Chen, B. Tian, G. Tan, M. Xu, Y. Qu, Y. Zhao, Y. Shang, and C. Tan, "Metarca: A generalizable root cause analysis framework for cloud-native systems powered by meta causal knowledge," 2026. [Online]. Available: <https://arxiv.org/abs/2603.02032>
- [35] Y. Luo, J. Jiang, J. Feng, L. Tao, Q. Zhang, X. Wen, Y. Sun, S. Zhang, and D. Pei, "Opsagent: An evolving multi-agent system for incident management in microservices," 2026. [Online]. Available: <https://arxiv.org/abs/2510.24145>
- [36] P. Di, F. Chen, X. Bai, H. Yang, Q. Li, G. Wei, J. Mou, F. G. Shi, K. Chen, P. Tang, Z. Shen, Z. Li, W. Shi, J. Guo, and H. Yu, "Openderisk: An industrial framework for ai-driven sre, with design, implementation, and case studies," *ArXiv*, vol. abs/2510.13561, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:282102496>
- [37] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604.
- [38] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying llm-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025.